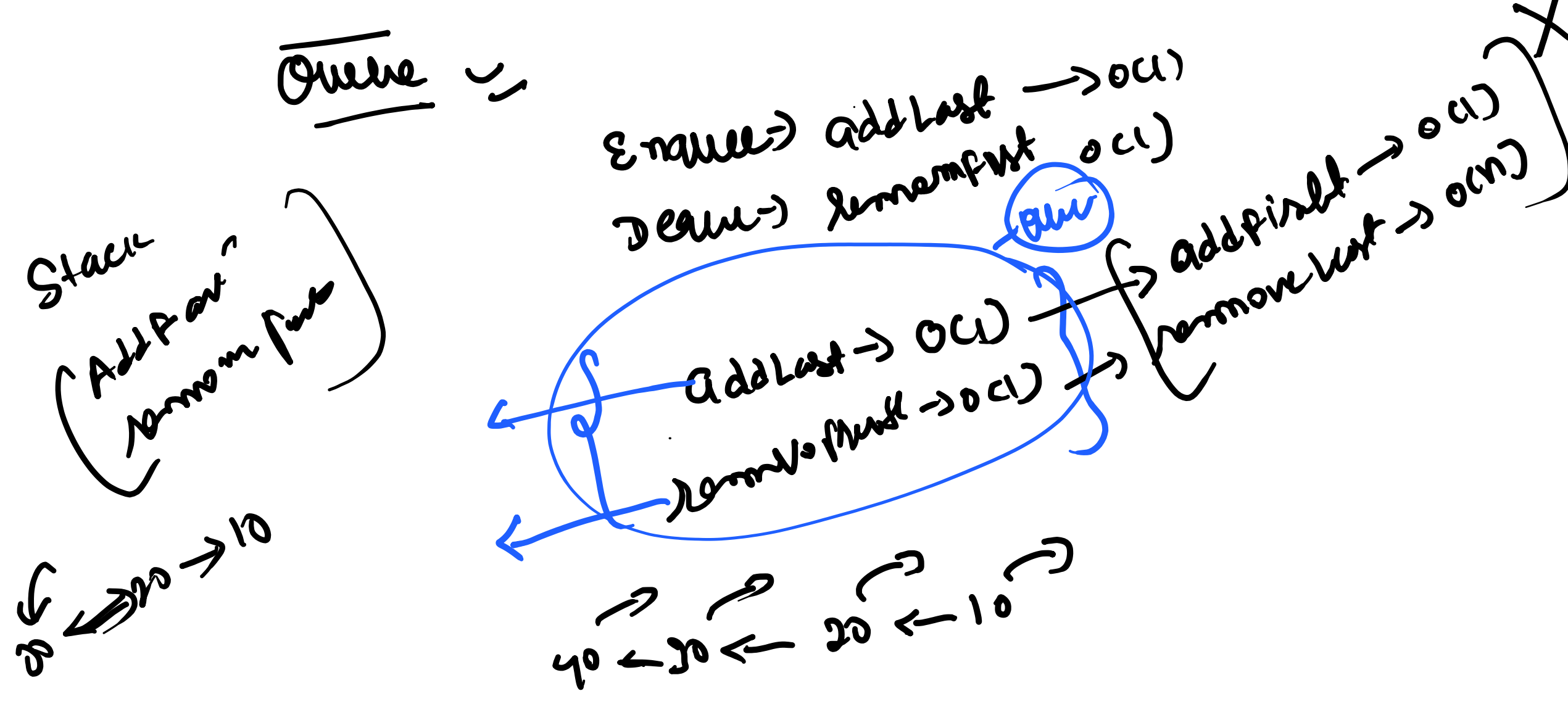
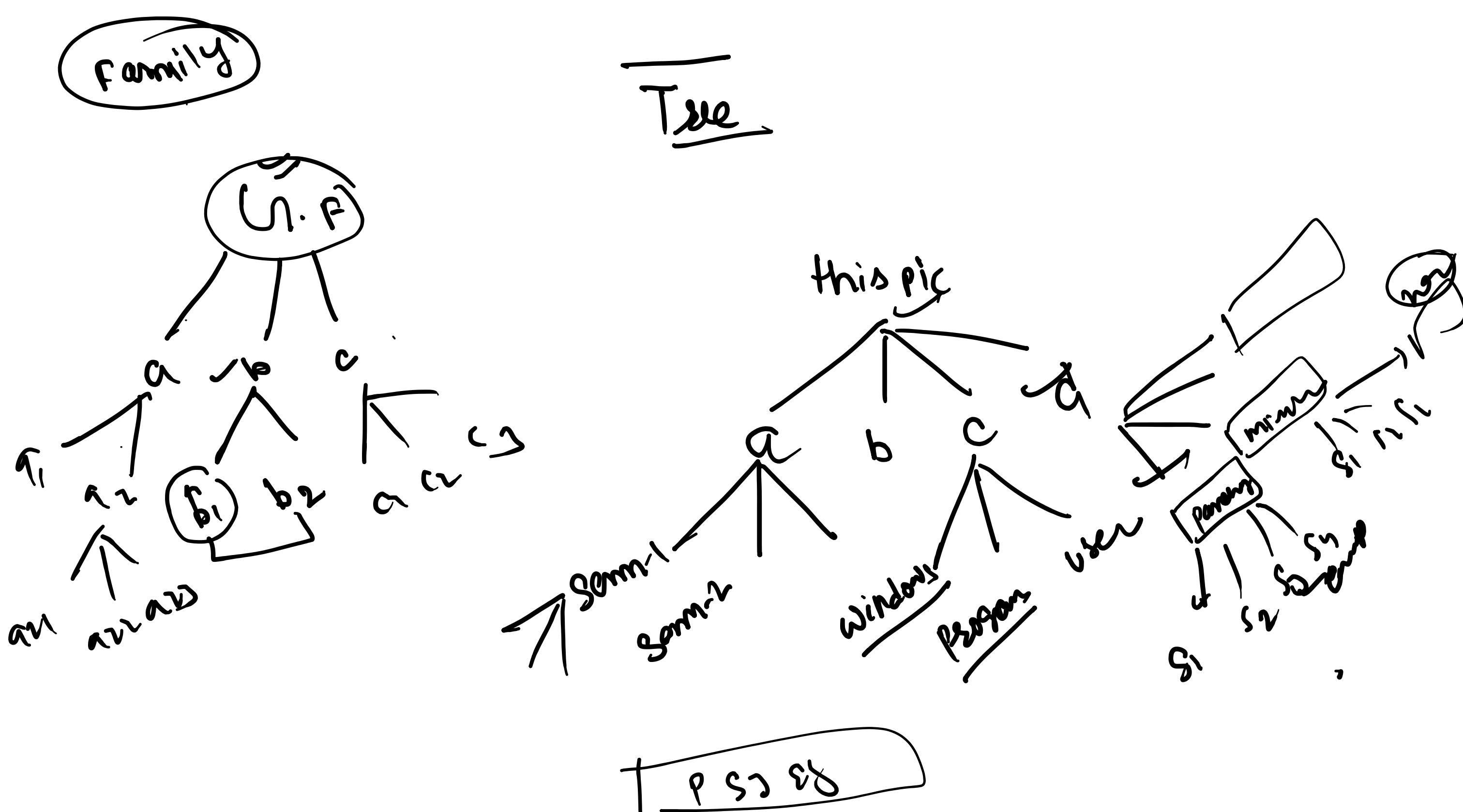
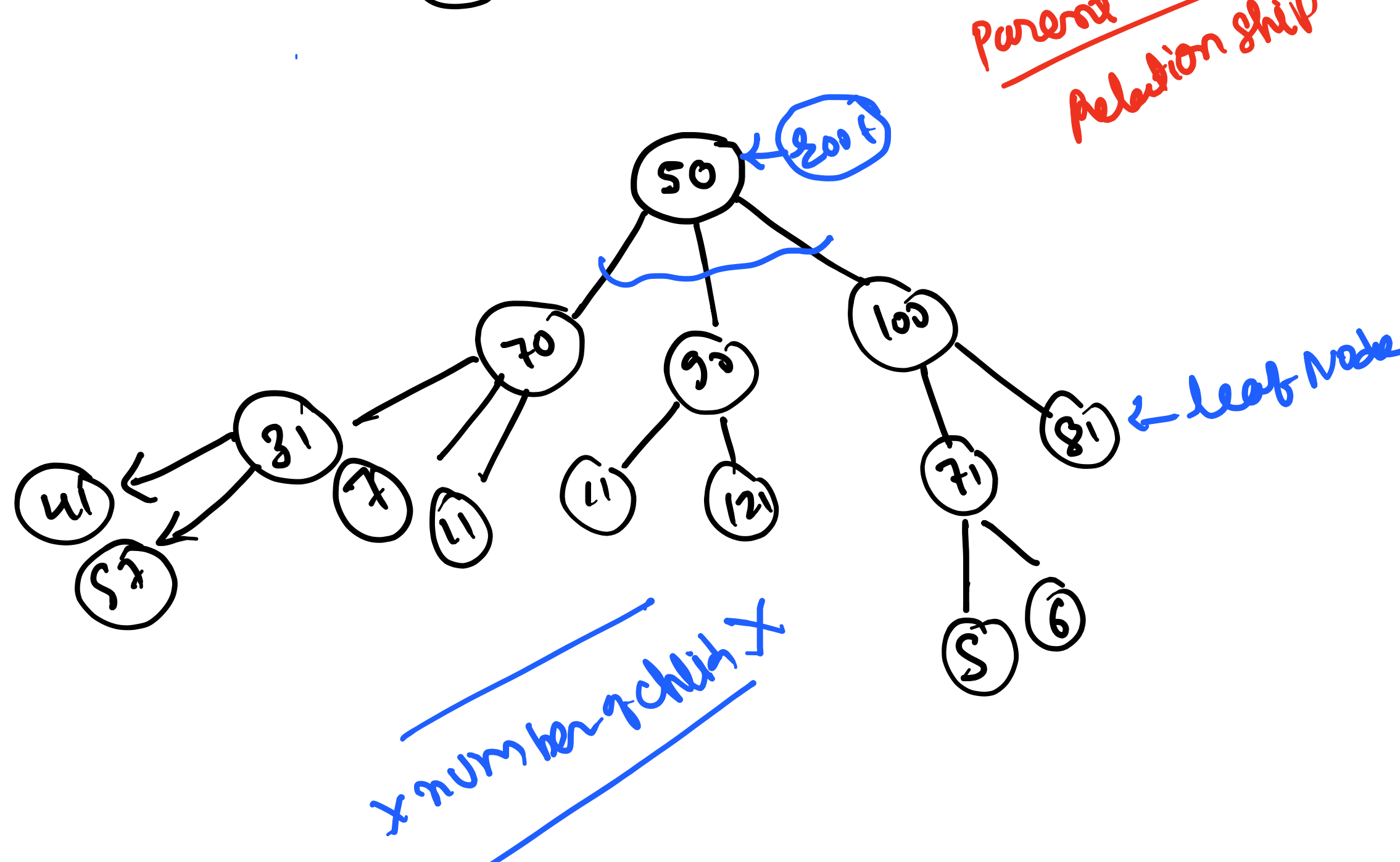




Tree non linear data structure



- 1) Chromatic Tree
- 2) Binary Tree
- 3) Binary Search Tree
- 4) Prefix Tree

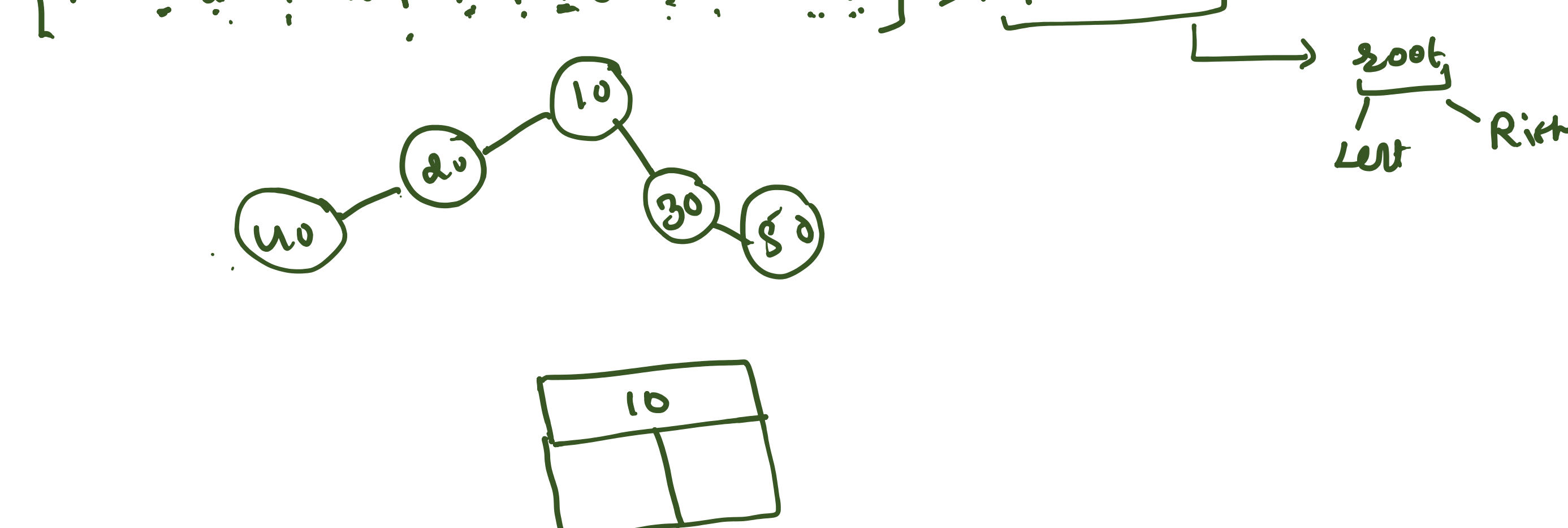
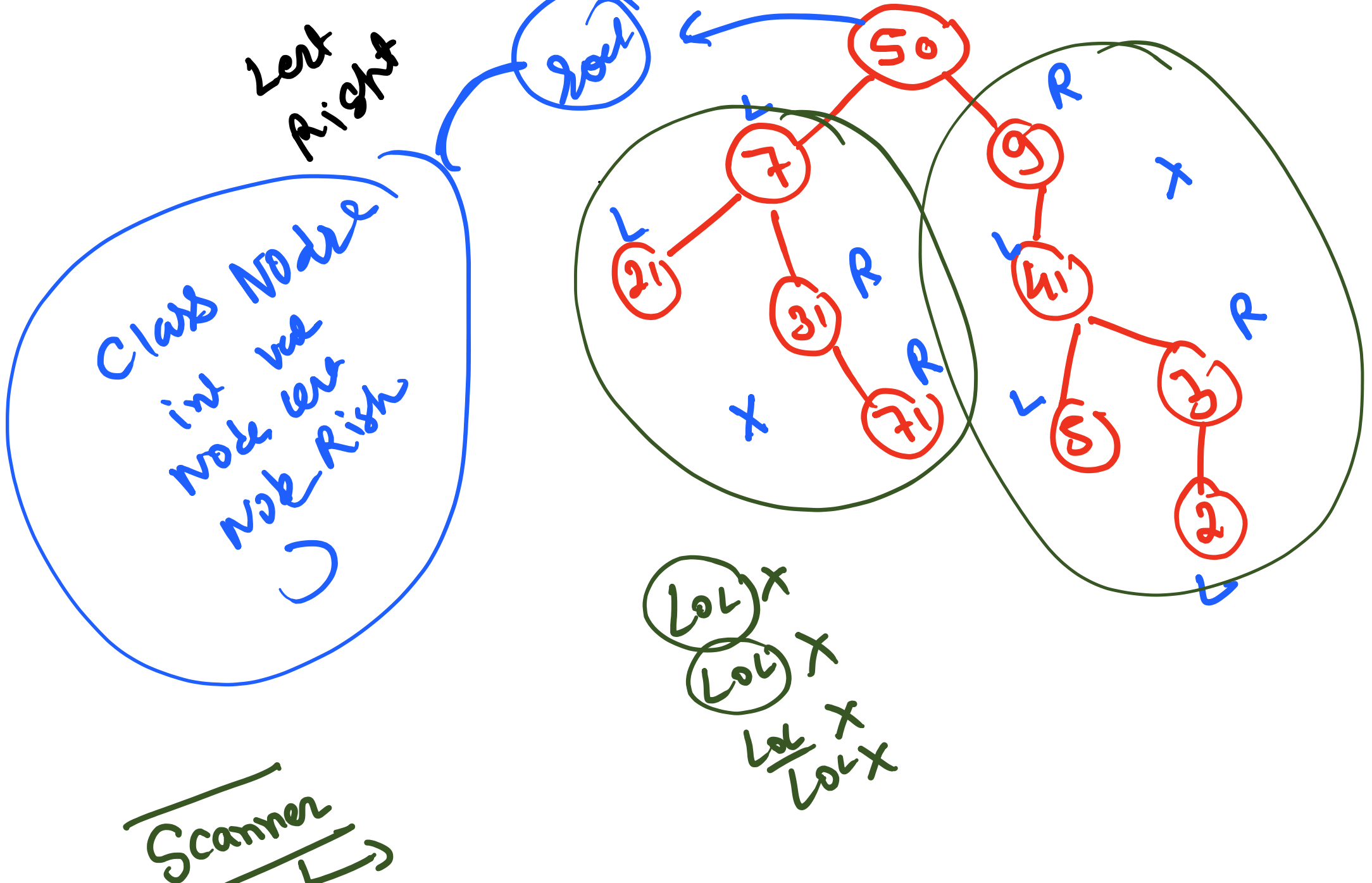


Binary Tree Binary Search Tree

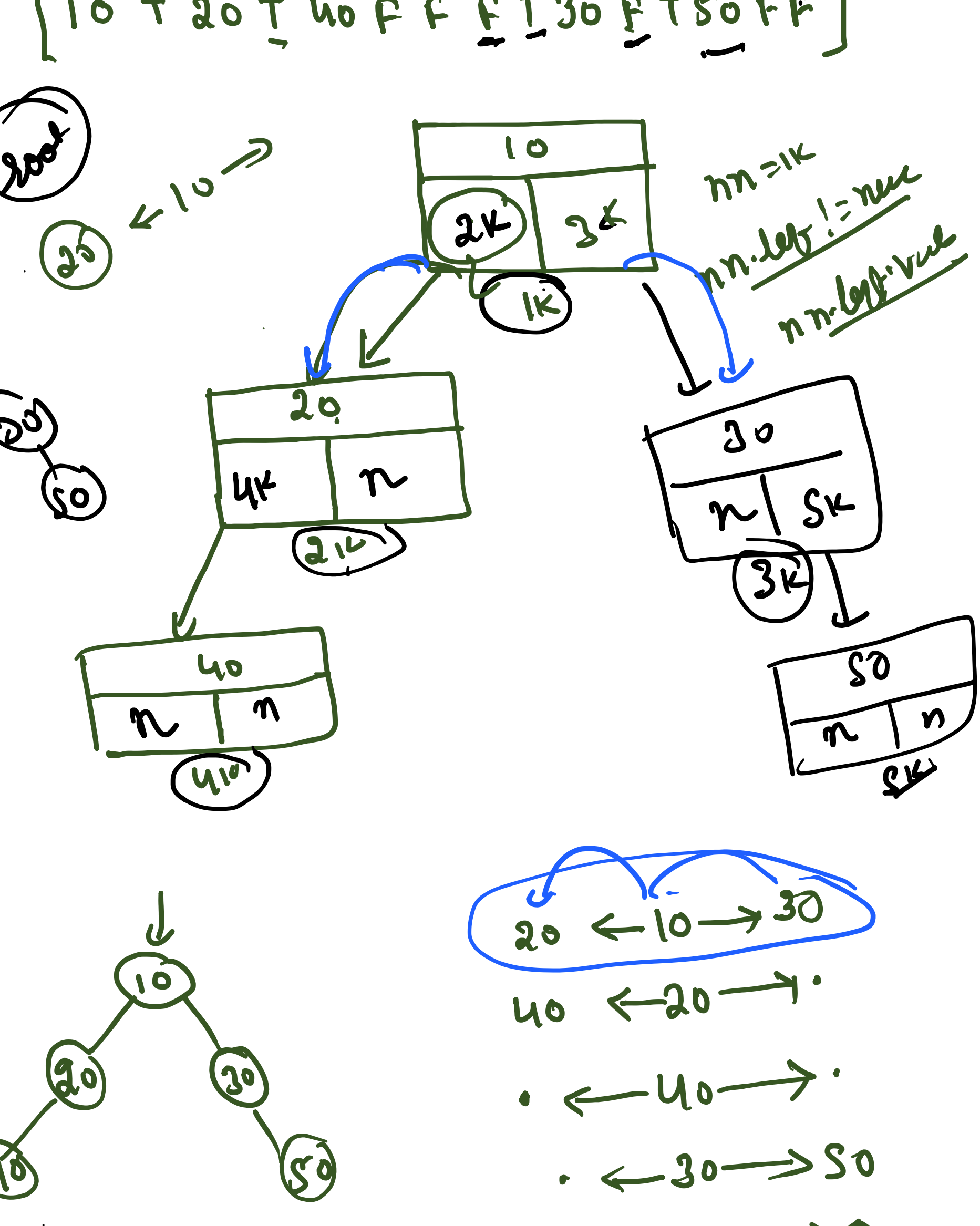
एक ही जगह की अधिकतम

की वजह से ही ऐसा करने

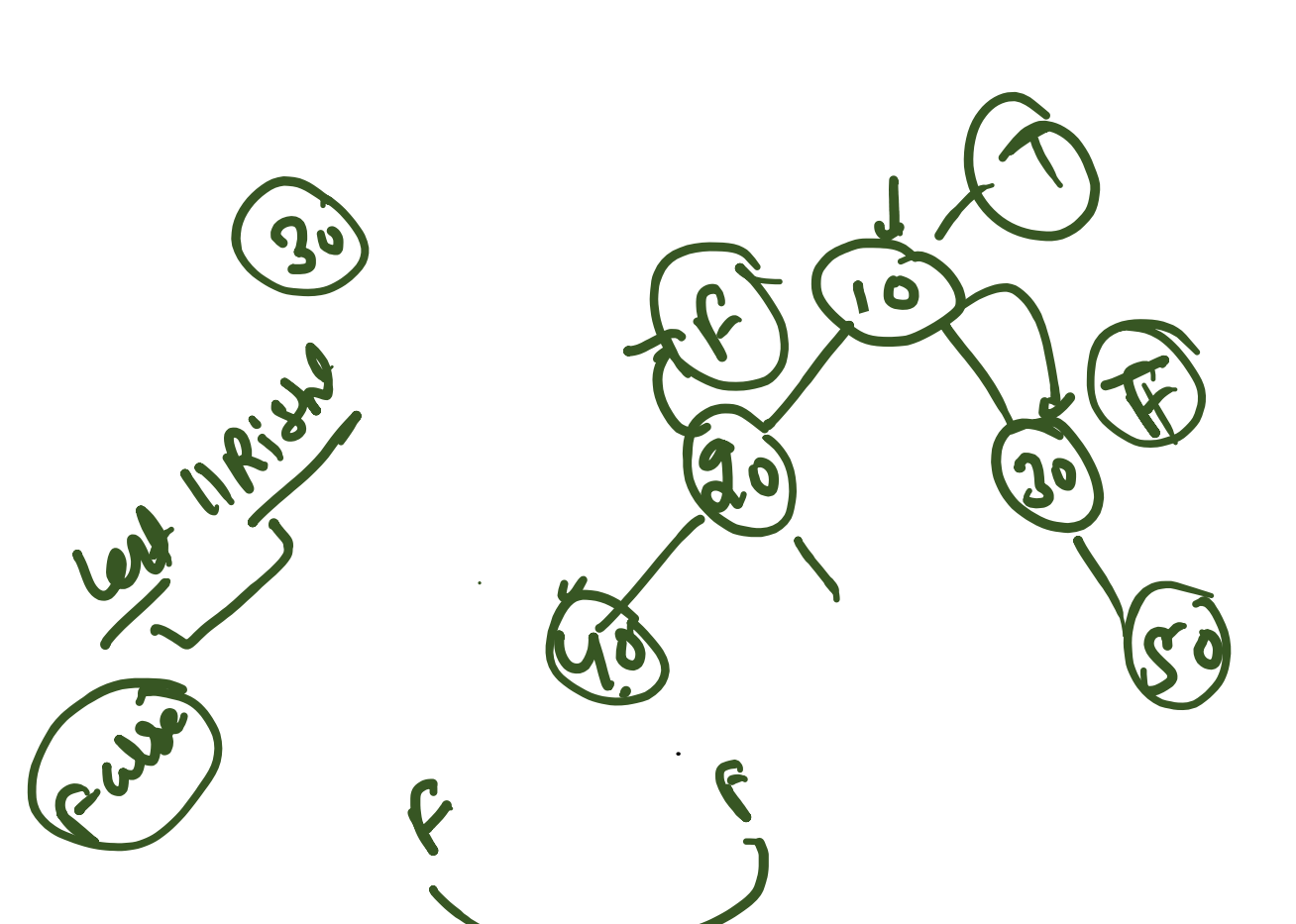
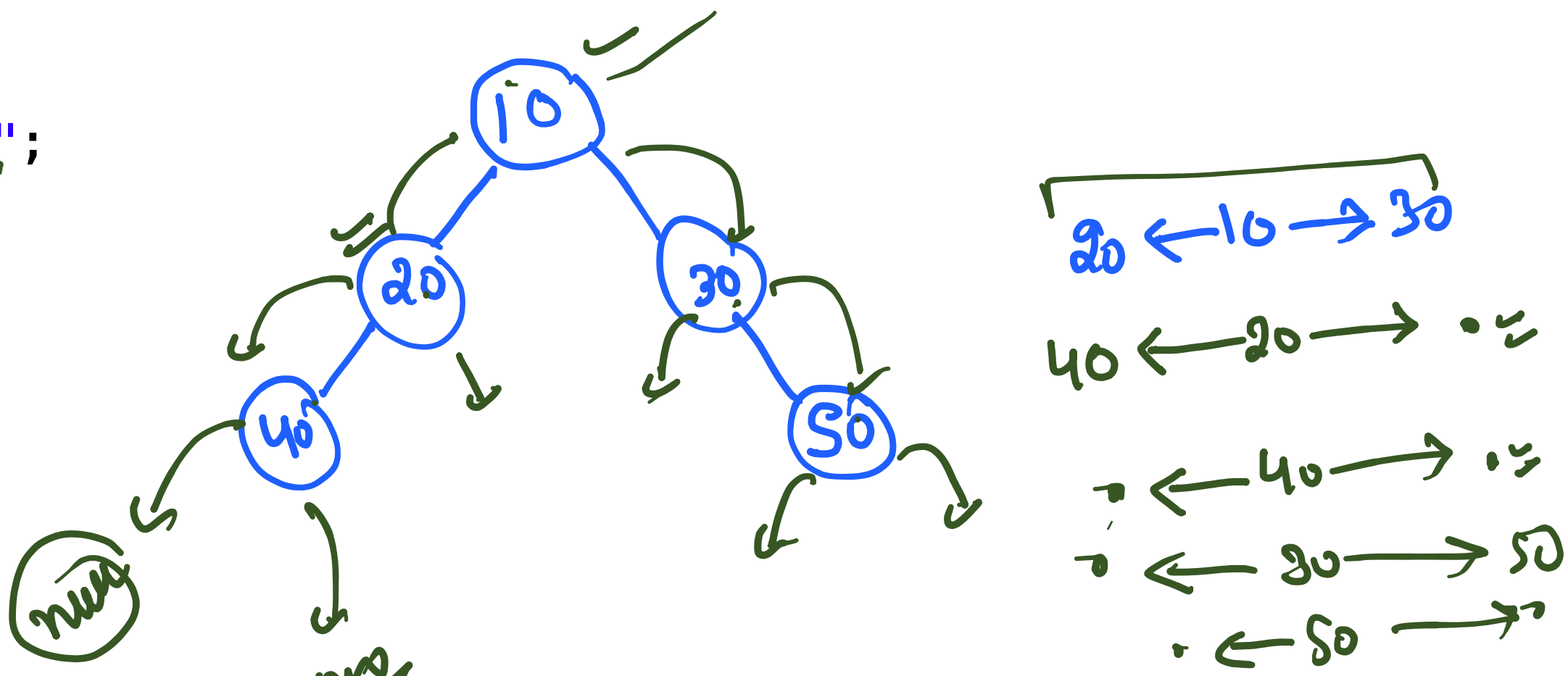
अवसर



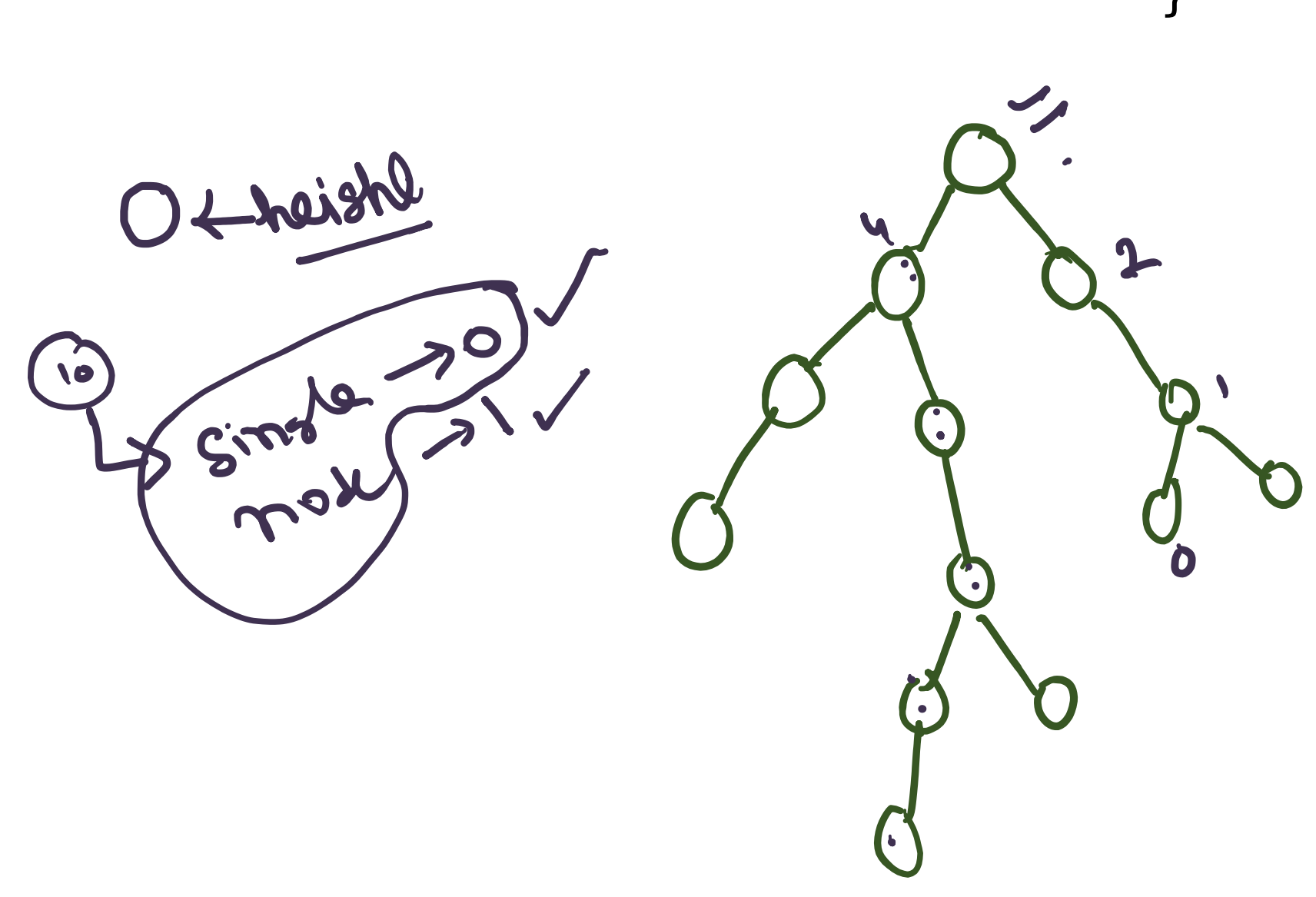
```
private Node CreateTree() {
    // TODO Auto-generated method stub
    int val = sc.nextInt();
    Node nn = new Node(val);
    boolean hlc = sc.nextBoolean();
    if (hlc) {
        nn.left = CreateTree();
    }
    boolean hrc = sc.nextBoolean();
    if (hrc) {
        nn.right = CreateTree();
    }
    return nn;
}
```



```
public void Display(Node nn) {
    String s = "<" + nn.val + ">";
    if (nn.left != null) {
        s = nn.left.val + s;
    }
    else {
        s = "." + s;
    }
    if (nn.right != null) {
        s = s + nn.right.val;
    }
    else {
        s = s + ".";
    }
    System.out.println(s);
    Display(nn.left);
    Display(nn.right);
}
```



```
private boolean find(Node node, int item) {
    // TODO Auto-generated method stub
    if (node == null) {
        return false;
    }
    if (node.val == item) {
        return true;
    }
    boolean left = find(node.left, item);
    boolean right = find(node.right, item);
    return left || right;
}
```

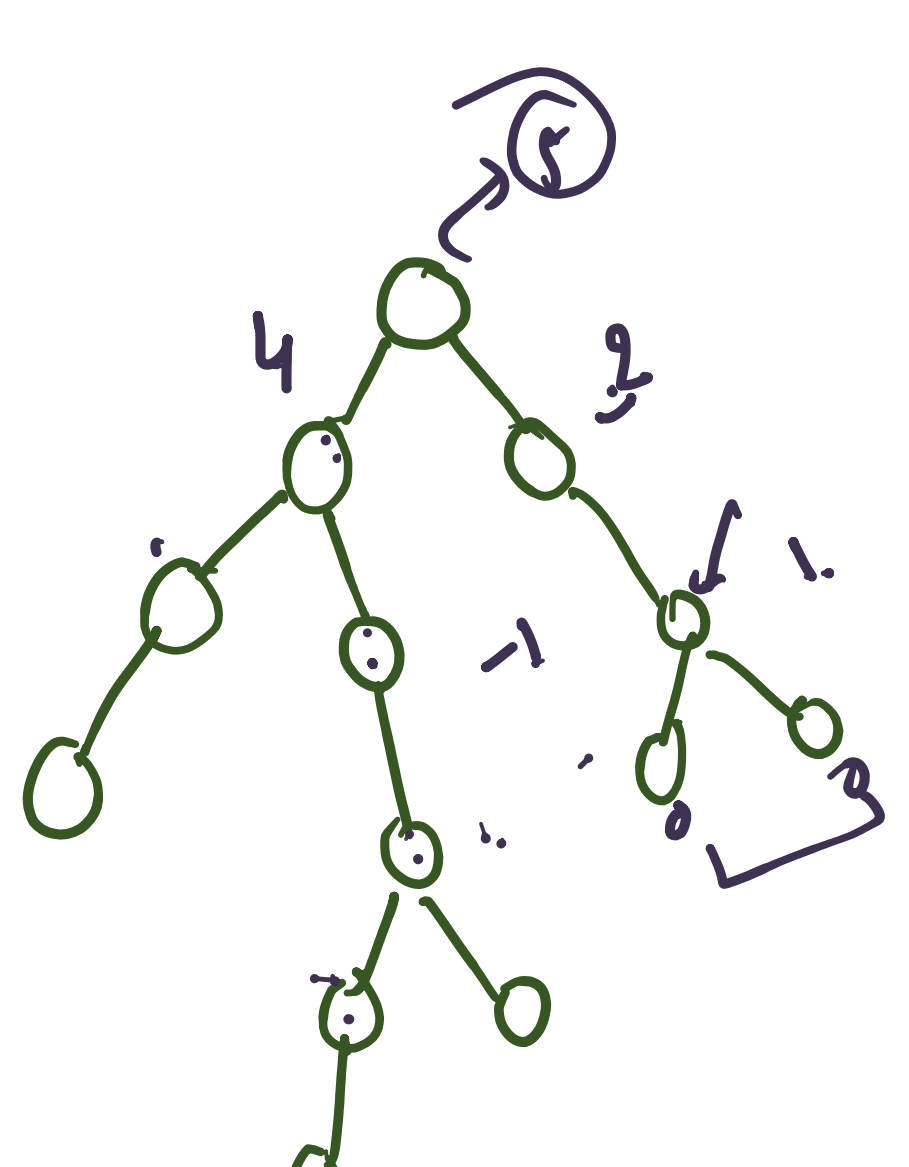


maximum distance b/w root node to leaf node is call height BT

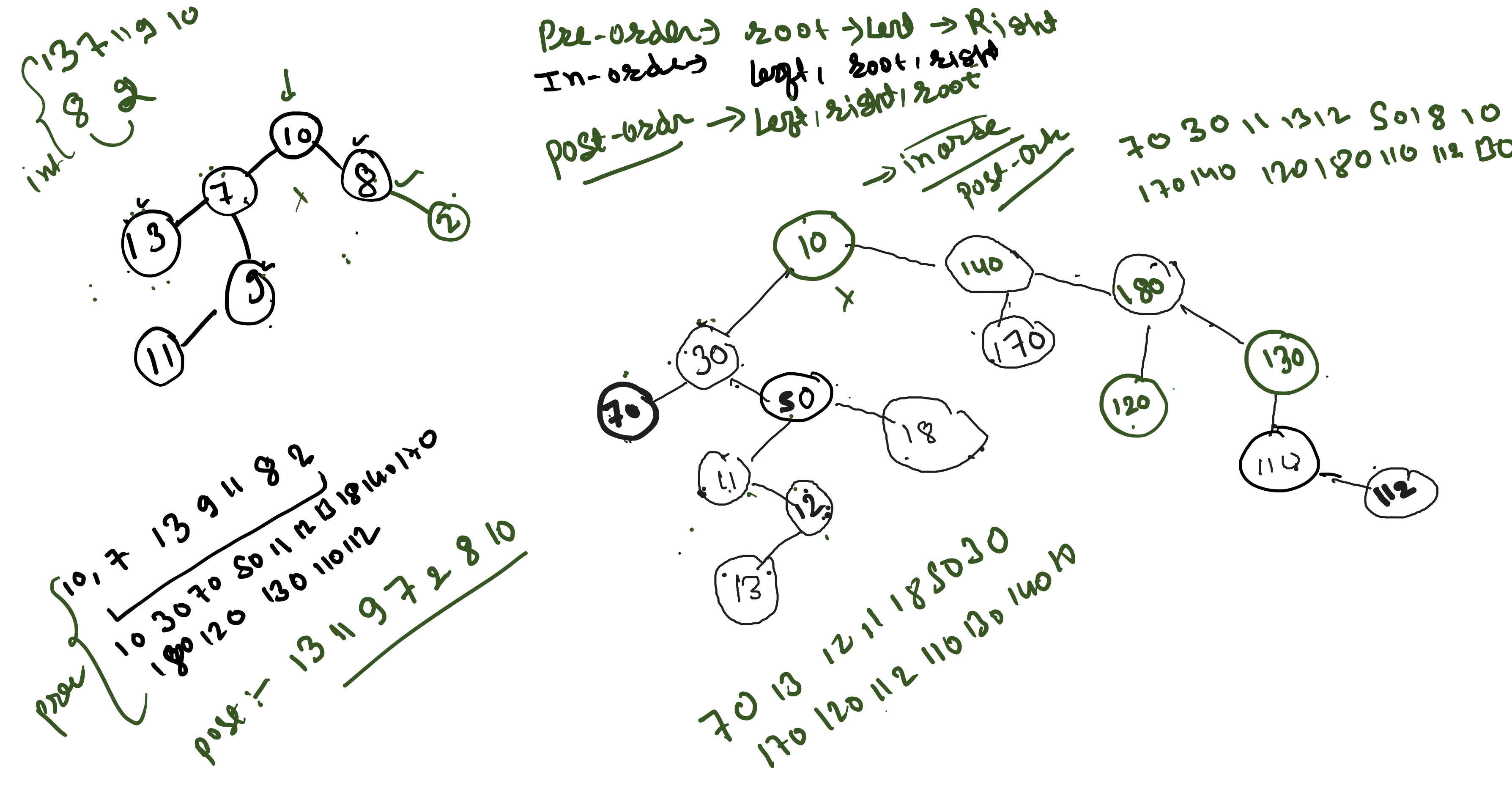
(Left ht, Right ht) + 1

(40) = 4 + 1 = 5

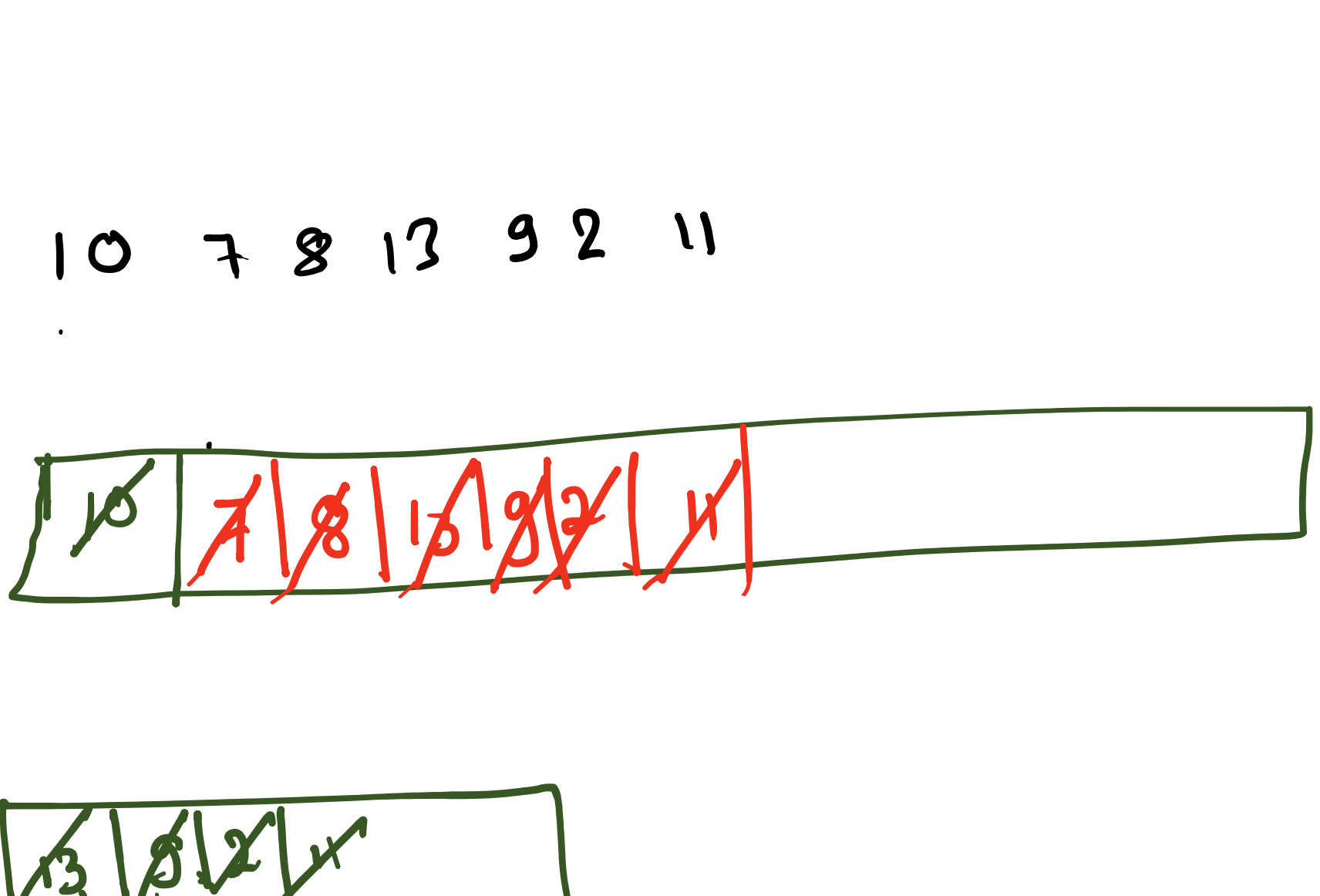
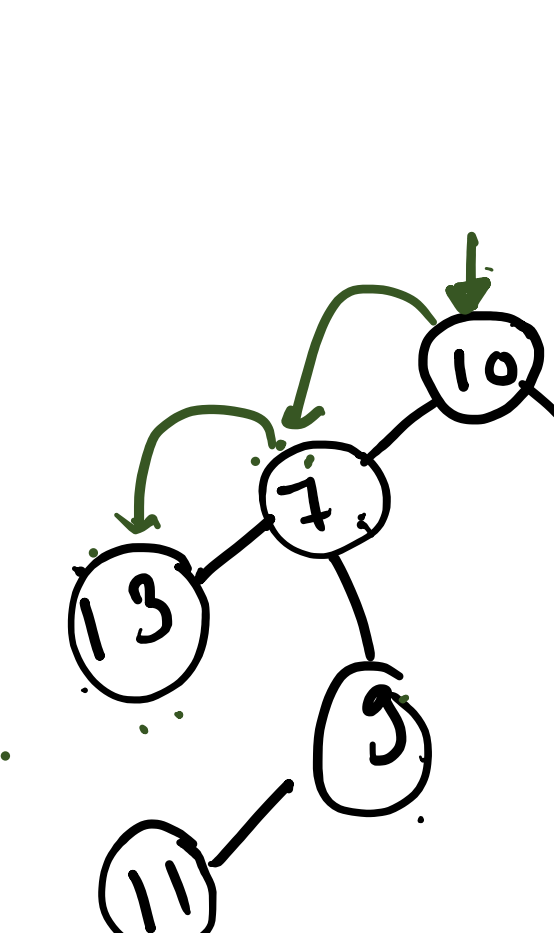
```
private int ht(Node node) {
    // TODO Auto-generated method stub
    if (node == null) {
        return 0;
    }
    int lh = ht(node.left);
    int rh = ht(node.right);
    return Math.max(lh, rh) + 1;
}
```



- 1) Pre-order
- 2) In-order
- 3) Post-order
- 4) Level-order



```
public void LevelOrder() {
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        Node r = q.poll();
        System.out.print(r.val + " ");
        if (r.left != null) {
            q.add(r.left);
        }
        if (r.right != null) {
            q.add(r.right);
        }
    }
    System.out.println();
}
```



10 7 8 13 9 2 11